

From Properties to Proof Obligations

A critical synthesis of the nine round-3 proposals

Central recommendation. Make mathematical properties available at the point of use through a small, proof-producing elaboration interface. Keep the objects and their meaning fixed; make the missing premises visible; use the same mathematical libraries and services in the Lean comparison.

The nine reports substantially agree on this direction. Their most valuable differences concern how inference is bounded, how observations justify both acceptance and rejection, and which mathematical interface should be implemented first. This report evaluates those differences, incorporates the parallel *Nine Refinements* review, and adds fresh kernel checks.

Prepared for Vladimir Reshetnikov

Codex | 6 September 2026

Self-contained: no earlier report is required. Input snapshot: 58bb54219f494b29310ff77384732129411da887. The seven supplied Lean specimens compile unchanged in this review; nine Python companions are reproduced. Those results validate particular artifacts, not an implemented language, a verified general CAS, or a measured productivity gain.

Abstract

Lean can express the relevant mathematics, but authors repeatedly connect an object to its known properties, the structures that give those properties meaning, and the premises of the next operation. Basalt, Fiber, Gneiss, Karst, Moraine, Obsidian, Schist, Tephra, and Trellis independently propose making this connection an explicit part of source-level typing and elaboration. Their common architecture is conservative: local refinements retain ordinary Lean objects, behavioral contracts describe operations relationally, and successful inference produces ordinary proofs. The strongest contributions go beyond notation: finite grounded inference, realizable abstract counterexamples, exact-quotient transport, coherent finite observations, support-sensitive proof methods, and denotationally justified checking.

This synthesis develops a common semantic model, compares the proposals rather than tallying their vocabulary, and works through representative mathematical arguments. It corrects a missing finite/free hypothesis in one exposition, separates theorem truth from statement and method fidelity, and updates the evidence by compiling all seven supplied Lean reference files unchanged. In particular, a small affine checker and a list-length interpreter now have fresh kernel-acceptance evidence. Two additional generic interface theorems are also checked. This revision incorporates the parallel review’s Frey proofs, separates four guarantees often conflated as abstraction completeness, and adds concrete tests for rewriting and incremental elaboration. The proposed next iteration should implement and compare a narrow property-aware interface on top of these artifacts, with explicit exit tests, instead of expanding the language design without an authoring experiment.

Contents

1	Executive assessment	3
1.1	The common answer, and the question still open	3
1.2	What the third round adds	3
1.3	Recommended decision	4
2	Corpus, provenance, and standards of evidence	4
3	Where the reports converge and where their choices differ	5
3.1	Ten shared commitments	5
3.2	The live design choices	5
3.3	Distinct contributions worth retaining	6
4	A common semantic core	7
4.1	Four entities, with different responsibilities	7
4.2	Local refinements and exported packages	7
4.3	Selecting structures is a separate decision	8
4.4	Behavioral contracts and their variance	8
4.5	What conservative elaboration actually promises	9
5	Inference that can be predictable	9
5.1	Open predicates, finite automation	9
5.2	Reconciling backward demands and forward closure	10
5.3	Cycles, branches, and changes to the context	10
5.4	A useful additional interface: the obligation frontier	10
6	Observations, constructions, and exact meaning	11
6.1	A relation is an interface, not a confidence level	11
6.2	Commuting observations and sufficient demands	11
6.3	Four guarantees behind finite evidence	11
6.4	Normalization is a construction	12
7	Worked case I: exact quotients without a vacuous benchmark	13
7.1	The theorem and the information it really needs	13
7.2	A reusable checked arithmetic package	13

7.3	A proposed authoring trace	14
7.4	Why the surrounding contradiction matters	14
8	Worked case II: finite observations of infinite operators	15
8.1	The behavioral hypothesis that makes restriction lawful	15
8.2	Negative neighbors reveal the exact scope	15
8.3	A reusable interface theorem added by this synthesis	16
9	Worked case III: analytical contracts and strength of conclusions	16
9.1	Orbital comparison without unnecessary hypotheses	16
9.2	Local equality, null boundaries, and quantitative observations	17
9.3	Summability licenses an interchange	17
9.4	Upper bounds are not optimality claims	17
10	Relational interfaces and a correction to the corpus	18
10.1	Contracts belong to the whole configuration	18
10.2	Finite indexing does not imply finite-dimensional factors	18
11	Leant: from candidate identity to proved behavior	19
11.1	The current boundaries deserve an accurate baseline	19
11.2	Why the bare function type is too weak	19
11.3	A concrete abstract interpretation: list lengths	20
11.4	Positive evidence and negative evidence need different bridges	20
11.5	Polymorphism does not supply parametricity for free	21
11.6	The synthesis experiment that would matter	21
12	Certified computation: what is now actually checked	21
12.1	The general proof chain	21
12.2	Schist closes one deliberately small chain	22
12.3	Moraine supplies a different, complementary link	22
12.4	Existing tools and execution policy remain controls	22
13	An implementation that tests the authoring hypothesis	23
13.1	Reuse Lean's state; add an interface to its evidence	23
13.2	Evidence after rewriting and incremental elaboration	23
13.3	Freeze meaning while allowing proof work to continue	24
13.4	Four small exit gates	24
13.5	Three acceptance checks, with separate failure states	25
13.6	When a kernel change would be justified	25
14	Evaluation that can change the design decision	26
14.1	Separate the causes of improvement	26
14.2	A workload portfolio with different failure modes	26
14.3	A compact adversarial acceptance suite	27
14.4	Semantic reading and stopping rules	28
15	Questions for the next iteration	29
A	Convergence crosswalk	30
B	Earlier questions, restated with current answers	30
C	Validation and reproducibility	32
C.1	Retained and fresh executions, with their limits	32
C.2	The parallel review and this revision's additional checks	32
C.3	What the package verifier checks	32

1 Executive assessment

1.1 The common answer, and the question still open

All nine proposals recommend improving the *type system the author uses* before changing the trusted kernel. A refinement is a proved property of a fixed term; a behavioral contract is a theorem about what a fixed operation requires and establishes. An elaborator can use these specifications to insert proof arguments, compose operations, and explain blocked applications. Lean’s dependent types already express the resulting objects and proofs.

For example, after introducing a positive real number x , a later division may need $x \neq 0$. The proposed layer should obtain that consequence from the existing positivity proof and retain the reason. It should not create a new real number, choose a different multiplication, or silently strengthen the theorem’s assumptions. The same idea scales to continuity on a particular region, preservation of an ideal, or the length behavior of a particular program.

This is a credible research direction. It is not yet evidence that a separate language is needed. Ordinary Lean already supplies local hypotheses, bundled structures, typeclass inference, tactics, extensible syntax, and substantial property automation. The decisive question is whether a unified use-site interface reduces authoring and repair effort after the baseline receives the same new theorems and services.

1.2 What the third round adds

The earlier computational boundary can be stated without reading either earlier synthesis: for a fixed request about an object a , accept a returned value b only with evidence for the requested relation $R(a, b)$. Equality, approximation, a solution witness, and a complete solution set are different relations. This round makes those contracts active *before* the result is produced, during application and synthesis. Its useful additions are:

1. **Properties without replacement.** Local evidence can enrich an existing object’s usable interface without repeated wrapping or a global instance for every proposition.
2. **Behavioral composition.** An operation states which input facts license its output guarantee, including relations between several objects and dependencies on actual intermediate witnesses.
3. **Predictable automatic fragments.** Arbitrary predicates remain expressible, while a finite, grounded rule layer offers a precise termination and relative-completeness claim.
4. **Two directions of abstraction.** A sound observation can support positive proof while an abstract counterexample still needs realization as an admissible concrete input before it supports rejection.
5. **More discriminating examples.** Exact integral coefficients, probability uniqueness, finite restrictions, ideal-preserving maps, and normalization expose obligations that cosmetic syntax changes would miss.

These are design developments relative to the shared brief, not claims of historical invention. Refinement types, contracts, abstract interpretation, predicate transformers, and proof-producing automation have substantial prior history. Correlated agreement among reports using the same material is a useful design signal, not nine independent usability experiments.

1.3 Recommended decision

Build a small property-aware interface over Lean, with ordinary Lean export, using two already checked pieces from this corpus: Schist’s affine-expression checker as a complete but tiny computation boundary, and Moraine’s list-length interpreter as a behavioral bridge. Exercise the interface first on a satisfiable exact-quotient problem and a small length-contract request. These test different claims; neither should be substituted for the other.

The first interface needs a binder for fixed objects and proved properties, theorem-backed requirement rules, visible unresolved obligations, and accepted proof terms. It does not need an independent mathematical database, a universal relation lattice, a general CAS, or a new kernel conversion rule. If the useful result is a Lean extension and better libraries, that meets the mathematical objective.

2 Corpus, provenance, and standards of evidence

The initial synthesis followed a fast-forward from `a18e3eb` to fetched `main` at `58bb542`. This revision then fast-forwarded from `856b619` to `8ddc6ab`, importing the parallel review *Nine Refinements* and its artifacts [12]. The nine original round-3 packages contain 83 tracked artifacts and 334 PDF pages. All nine TeX sources were read in full across three parallel review lanes; their examples, companions, source registers, and claims were examined critically. The original packages remain unchanged.

Proposal	Pages	Most useful distinguishing emphasis
Basalt	40	Relational diagrams, ideal-preserving completions, and scoped probability facts.
Fiber	38	Exact quotient transport, inhabited arithmetic controls, and quantitative coupling.
Gneiss	37	Finite-section inversion and the separation of representation from behavior.
Karst	40	Support-indexed evidence and finite-set reasoning without silently choosing enumeration.
Moraine	36	Property rows, higher-order interfaces, and a concrete list-length semantics.
Obsidian	34	Witness-changing normalization, satisfiable helper contracts, and a stronger Lean baseline.
Schist	33	Backward requirements, orbital comparison, and a small sound affine checker.
Tephra	39	Observation coverage, prefix-local inverses, and realizable negative evidence.
Trellis	37	Finite qualifier closure, explicit accounting, and rational inequality certificates.

The two revised round-2 syntheses are background sources, not additional round-3 votes. Their relevant concepts and open questions are restated here. The new input includes further revisions of the secondary round-2 report; the current source register binds the versions actually imported, rather than reusing the previous review’s identities. References to ProveIt and the Fermat repository are primarily source studies made by the proposals. This review does not claim to have rebuilt or audited either development.

The parallel round-3 synthesis is a secondary reading of the same material, not a tenth independent proposal. Its 28 artifacts have a separate immutable register. Its Frey proof, operational questions, and extended negative-test catalogue inform this revision. Its 76-commitment matrix uses finer editorial criteria than our ten dimensions; those counts cannot be pooled as votes. Analogous semantic combinators also do not make different Lean structures, subtypes, interpreters, and checkers literally alpha-equivalent programs.

We distinguish four kinds of evidence throughout:

Design or written proof

A specified interface or mathematical argument. It can be valuable and correct without being

implemented or kernel checked.

Executed finite model

A Python program passed particular finite checks. This does not prove a generic algorithm or its correspondence to Lean.

Fresh Lean acceptance

A specified file was accepted by a pinned Lean compiler; selected declarations have retained axiom inventories. This does not verify a frontend absent from that file.

Authoring evidence

Measured time, semantic comprehension, or maintenance cost under a controlled comparison. None of the nine proposals supplies this.

The source register, reading memos, per-report convergence records, execution receipts, and PDF validation accompany the report. A page count is PDF metadata; it does not establish that an input PDF was rebuilt from its TeX here. Report citations credit the proposed ideas; the machine-readable crosswalk supplies precise source-line anchors. The comparison is editorial reading with visible qualifications, not an automated novelty score.

3 Where the reports converge and where their choices differ

3.1 Ten shared commitments

The evidence map compares ten broad commitments: conservative elaboration; same-object scoped evidence; separation of structures from propositions; dependent behavioral contracts; bounded inference; justified observations; proof-linked synthesis; checker semantics and target reconstruction; distinct truth/fidelity obligations; and a matched evaluation with stopping rules. The appendix and accompanying JSON keep the per-proposal evidence and qualifications. The categories describe policy commitments, not completed implementations or ten equally difficult engineering features.

The strongest convergence is therefore architectural. The reports reject arbitrary theorem proving inside definitional equality, properties inferred from attractive names, unproved provider laws promoted to universal behavior, and a generic “verified” status that hides the exact guarantee. They also converge on retaining imported Lean semantics and on allowing explicit partial-domain packages for new mathematics.

3.2 The live design choices

Choice	Competing emphases	Assessment
Inference strategy	Schist emphasizes backward demands; Trellis gives full closure over a finite grounded vocabulary.	Combine bounded rule generation with complete closure inside a backward-closed relevant fragment; expose incompleteness outside it.
Local representation	Contextual overlays retain x ; exported subtypes or structures package x and its proofs.	Use overlays locally and bundles at explicit API boundaries. These are complementary representations.

Choice	Competing emphases	Assessment
First computation	Several reports specify polynomial ideal certificates; Trellis favors affine inequalities; Schist supplies a tiny affine equality core.	Start with the checked tiny core. Select a larger checker because a workload needs it, not because all CAS operations should share one implementation.
First behavioral slice	Moraine’s list DSL is concrete; other reports emphasize broader structural providers and abstract Length laws.	Prove the restricted interpreter link first; require proof linkage for each added provider and exact source reconstruction.
First mathematical case	Exact division, regularity, finite inverses, probability, and categorical constructions each expose different pain.	Use exact division for the first end-to-end interface and a nonalgebraic transfer task next. Preserve already-short Lean controls.
Kernel research	Native refinements or stronger conversion remain research alternatives.	Require an isolated measured obstruction before changing the trusted foundation. A verbose wrapper is not by itself such an obstruction.
Proof-method control	Some interfaces treat a named reason as a route hint; all acknowledge method fidelity as distinct from theorem truth.	Make hint versus required-method status explicit. A final proof can be true while violating the requested argument.

There is less disagreement about foundations than the nine names suggest. The choice is chiefly which integration hypothesis to test first and how much automatic behavior to promise. A synthesis should not invent incompatible camps from differences of emphasis.

3.3 Distinct contributions worth retaining

Basalt makes the strongest case that a mathematical property interface cannot be a row of unary adjectives. Exactness involves several maps; a map of completions needs compatibility at every quotient level; probability symmetry needs the correct measure and null-boundary condition. Its warning about realizable abstract inputs connects these examples to synthesis [1, Sections 10–15].

Fiber turns the Frey coefficient example into a genuinely informative helper contract by removing the surrounding contradiction assumptions. It separates exact integer division, homomorphic preservation of a balance equation, and invertibility in the destination. Its probability example distinguishes almost-everywhere, distributional, and quantitative conclusions rather than calling them all convergence [2, Sections 9–10].

Gneiss treats finite presentation as a behavioral operation, using prefix locality to justify finite square sections. It also makes large proof assembly a useful source of contracts while refusing to count generated environment setup as mathematical verbosity. Its finite-section argument pairs well with Tephra’s observation interface [3, Sections 3, 8, 10].

Karst emphasizes the actual support of evidence and the difference between a finite type and a chosen executable enumeration. Its finite-map example is an instructive control: common mathematics may be proved without silently installing additional computational data. Its `List Empty` example is now directly checked in the supplied Lean file [4, Sections 8, 12].

Moraine contributes the most concrete route from a tiny source grammar to a universal length statement. Its higher-order examples are valuable, but one finite/free conclusion needs hypotheses

restored (Section 10.2). The checked list interpreter is narrower than its Python coefficient normalizer and should be credited at that exact scope [5, Sections 6, 10].

Obsidian adds an essential category: normalization may construct a new witness, so it is neither learning a property of the old witness nor observing it. It also insists that the Lean baseline include existing `fun_prop` and program-verification support, and that helper tests remain informative outside a contradictory application context [6, Sections 2, 6, 12, 14].

Schist connects backward requirements to a reusable orbital-comparison argument without adding moment assumptions or cancelling a possibly zero multiplier. Its supplied affine checker is especially useful because this review can now promote its small semantic argument from uncompiled source to fresh Lean acceptance [7, Sections 7, 9, 16].

Tephra separates transfer, observation coverage, and negative realizability. It carefully distinguishes arbitrary modules from finite free prefix spaces and polymorphic typing from a parametricity theorem. These are precisely the distinctions that an ambitious inference layer could otherwise erase [8, Sections 7–8, 13].

Trellis offers the clearest bounded-inference theorem and explains its cost exclusions. Its affine-inequality certificate format matches actual bound obligations, and its accounting distinguishes the effect of a library from services and authoring syntax [9, Sections 7, 13, 18].

4 A common semantic core

4.1 Four entities, with different responsibilities

Let Γ be an ordinary dependent Lean context. It contains the selected types, objects, structures, and assumptions. A local evidence index may speed lookup and record explanations, but it is not a second logical context. The following four entities should remain distinct:

Entity	Mathematical content	What may change
Object	$a : A$, with fixed implicit structure arguments	Only through an explicit definition, choice, equality, or construction.
Refinement	A proof $p : P(a)$ in Γ	More proved facts can become available without replacing a .
Observation	A value $b : B$ with $R(a, b)$	The view or precision can change through a proved relation.
Construction	A new b with a relation $C(a, b)$	The witness changes; the preservation guarantees must be stated.

A fifth entity, an operational receipt, records execution, source identity, environment, and acceptance policy. It is useful engineering evidence but is not itself a mathematical proof of $P(a)$.

4.2 Local refinements and exported packages

Write the local judgment

$$\Gamma \vdash a : A \triangleright \{P_1(a), \dots, P_k(a)\}.$$

It means that $a : A$ and each listed proposition has a proof in Γ . There is no new kernel typing rule hidden in this notation. Introduction adds a proof; forgetting omits a property from the interface;

intersection combines proofs about the *same* term. A checked implication

$$h : \forall x : A, P(x) \rightarrow Q(x)$$

permits weakening from $P(a)$ to $Q(a)$ by applying h to the actual evidence. This is proof-producing subsumption, not definitional equality.

For an exported value, $\{x : A \mid P(x)\}$ or a structure with fields for x and its proofs can be appropriate. Repeatedly introducing such packages at every local use would recreate coercion and identity bookkeeping. The synthesis therefore recommends local views of fixed terms and explicit exported bundles, rather than choosing one representation for every purpose.

Properties may be relations such as $\text{Inverse}(f, g)$ or predicates indexed by a measure, action, region, or topology. A “row” is a convenient presentation of a family of propositions, not a restriction to unary records. Two instances of **Continuous** f under different topologies are different elaborated propositions even when a compact display looks identical.

4.3 Selecting structures is a separate decision

An order, topology, multiplication, action, or measure can determine the theorem being stated. A proof that a fixed multiplication is associative does not select that multiplication. Proof irrelevance equates proofs of the same proposition; it does not equate different operations or make arbitrary structures interchangeable.

Automatic reconstruction of meaning-bearing data is appropriate when the source already fixes it uniquely by an explicit convention or a specified construction. Ambiguous alternatives should remain a visible choice. Importing a new instance must not silently change the statement merely because the new interpretation makes its proof easier. The implementation should compare elaborated targets, including implicit structure arguments, when checking source stability.

4.4 Behavioral contracts and their variance

For a fixed total function $f : A \rightarrow B$, define

$$\text{Beh}(f; P, Q) := \forall x : A, P(x) \rightarrow Q(x, f(x)),$$

where $Q : A \rightarrow B \rightarrow \text{Prop}$. A total function with a conditional theorem is not a partial function. A function whose *input type* requires a proof of $P(x)$ is another interface; the source must say which one is intended.

Consequence has a direction. From $\text{Beh}(f; P, Q)$, $P' \Rightarrow P$, and $P'(x) \wedge Q(x, y) \Rightarrow Q'(x, y)$, derive $\text{Beh}(f; P', Q')$. The new client supplies a stronger precondition and accepts a weaker conclusion. For behavioral replacement, the implementation offered to a caller must require no more and guarantee no less than the caller’s contract. These are compatible statements viewed from opposite sides of the interface, not a license to reverse an implication.

Proposition 4.1 (Composition with the missing bridge made explicit). *Suppose $\text{Beh}(f; P, Q)$ and $\text{Beh}(g; U, V)$, and there are proofs*

$$\begin{aligned} P(x) \wedge Q(x, y) &\Longrightarrow U(y), \\ P(x) \wedge Q(x, y) \wedge V(y, z) &\Longrightarrow W(x, z). \end{aligned}$$

Then $\text{Beh}(g \circ f; P, W)$.

Proof. For x satisfying P , obtain $Q(x, f(x))$. The first bridge establishes the precondition for g at that intermediate value; its contract gives $V(f(x), g(f(x)))$. The second bridge yields the desired W . $\square$

This elementary theorem is the right model for a method registry. The bridge cannot be omitted just because the codomain of f matches the domain of g . If the second operation also depends on x or on the evidence produced at the first stage, use the corresponding dependent function and keep those bindings in order. A flat set of guard strings cannot represent this dependency.

4.5 What conservative elaboration actually promises

For a restricted source calculus, a useful preservation statement says: if a derivation is accepted with all generated obligations discharged, then its translation is a Lean term of the translation of the *specified* source type. The proof proceeds by induction over source rules: property introduction inserts its proof, consequence applies its theorem, and composition applies the bridge just described.

It is not enough to claim “whatever the frontend emits is kernel checked.” A frontend could emit a proof of the wrong proposition. The source semantics, translation of dependent contexts, and relation between the authored and elaborated statement are part of the claim. The supplied Lean files prove some rule encodings; none implements and verifies the proposed parser, name resolution, rule generation, or whole translation.

5 Inference that can be predictable

5.1 Open predicates, finite automation

Allowing arbitrary Lean predicates in refinements does not imply complete automatic entailment. A useful predictable fragment starts after the meaning-bearing terms have been elaborated. Fix a finite set $\mathcal{Q}$ of well-typed propositions in the current context and a finite set of ground rules

$$P_1 \wedge \cdots \wedge P_k \longrightarrow Q, \quad P_i, Q \in \mathcal{Q},$$

each backed by a theorem instance. “Ground” here means that the relevant objects and data parameters are fixed expressions; an arbitrary local real variable is allowed. It does not mean enumerating real numbers.

From the initial proved facts S_0 , repeatedly add a conclusion whose premises already have evidence. The resulting closure has four precise properties:

1. at most $|\mathcal{Q} \setminus S_0|$ new facts are added;
2. each new fact has a proof assembled from earlier evidence;
3. the result is the least rule-closed superset of S_0 ;
4. fair scheduling changes the selected proof terms, but not that fact set.

Termination follows from finiteness and monotonicity. Soundness follows by induction over additions. Every closed superset contains each addition, proving leastness; fairness eventually processes each enabled rule. Trellis supplies this clean formulation [9, Section 7].

With premise counters, the bookkeeping can be linear in the ground facts and premise occurrences. This excludes generating rules, typechecking theorem instances, normalization, unification, and proof

size. It is not a complexity bound for the entire elaborator or a principal-type theorem for arbitrary Lean.

5.2 Reconciling backward demands and forward closure

Schist’s demand-driven strategy and Trellis’s closure theorem fit together. Start with the requested propositions, construct a bounded set of relevant theorem instances, and retain all dependencies of every included rule. Complete closure inside that fixed backward-closed fragment is then predictable relative to the fragment. Heuristic pruning or exhausted rule-generation budgets can be useful, but forfeits any broader completeness promise.

For a total function f , a sufficient requirement transformer Req_f satisfies

$$\text{Req}_f(Q)(x) \implies Q(f(x)).$$

If Req_g is sound, then $\text{Req}_f(\text{Req}_g(Q))$ is sufficient for $Q(g(f(x)))$. There is no assertion that these are weakest preconditions. Offering two different sufficient routes is often better mathematics than forcing one global normal form of all hypotheses.

5.3 Cycles, branches, and changes to the context

Rules $P \rightarrow Q$ and $Q \rightarrow P$ do not create a fact without a seed. A proof of $G \rightarrow R$ cannot discharge G by using the conditional R as though it were already available. Recursive reasoning needs an independently justified induction or recursion principle; a cyclic cache entry is not that principle.

Likewise, evidence from one branch cannot leak into its sibling. Joining branches requires the corresponding case proof, even if both branch displays contain the same sentence. Reuse across contexts requires a typed substitution for old variables and proofs of the old assumptions under that substitution. Hashes and stable names help find possible matches; they do not prove the transport.

This suggests two separate records: a compact lookup index and an accepted derivation whose dependencies are Lean terms. Invalidation should follow the second record when meaning, assumptions, or accepted rules change. A failed bounded search should report an unresolved obligation and the available routes, not a negated mathematical proposition.

5.4 A useful additional interface: the obligation frontier

The report recommends making the frontier of unfinished proof work a first-class authoring view. Each open item should name the actual proposition, its local context, the operation needing it, and a small set of sufficient routes. For example, “need $g(x) \neq 0$ to use the quotient derivative; the available lower bound applies only on $[a, b]$ ” explains a mathematical obstruction.

An implementation need not find a smallest explanation, which may itself be expensive and unstable. It should provide a valid dependency slice and retain the proof terms when the frontier closes. Measuring repeated manual obligations removed by this interface is more informative than counting how many facts its index contains.

6 Observations, constructions, and exact meaning

6.1 A relation is an interface, not a confidence level

A result indexed by a has the schematic type

$$\text{Result}(a, R) = \{b : B \mid R(a, b)\}.$$

The relation can express exact representation, equality on a region, an interval bound, equality modulo X^N , or a complete solution predicate. These guarantees are not levels of confidence. They require different theorems for their use and do not form a single universally ordered ladder.

Three inherited corrections remain essential. A complete solution set S needs both $x \in S \Rightarrow P(x)$ and $P(x) \Rightarrow x \in S$; one good witness plus coverage allows extraneous solutions. An open set helps establish local equality only when the functions agree throughout the neighborhood, not merely at its center. An enclosure $\ell \leq x \leq u$ does imply equality when $\ell = u$. A registry should store precise implication theorems and their premises rather than categorical “always” or “never” promotions.

6.2 Commuting observations and sufficient demands

For an operation $f : A \rightarrow B$, an observation interface may supply

$$\text{obs}_B(f(x)) = \hat{f}(\text{obs}_A(x)).$$

This commuting-square theorem permits computation through the observation. It does not say that either observation is injective or covers every abstract input. Those facts must be separate when needed.

For precision-indexed observations, a demand function may require input order $d_f(N)$ to produce output order N . Composition uses $d_f(d_g(N))$. Differentiation of a formal series typically needs order $N + 1$ to establish order N of its derivative. An arbitrary finite prefix of a series is not the series itself, and finitely many checked prefixes do not replace a theorem quantified over every prefix.

6.3 Four guarantees behind finite evidence

The parallel review usefully asks when finite evidence can settle a universal claim, but its proposed completeness table combines distinct questions [12, Section 9.2]. An implementation should expose four separate contracts:

1. **Positive soundness.** Acceptance of a finite certificate implies the requested source proposition. A theorem $\text{check}(c) = \text{true} \Rightarrow \forall x, P(x)$ can justify a universal conclusion even if the checker fails to recognize many true claims. Completeness is unnecessary for that successful proof.
2. **Decision completeness.** Every true claim in a stated candidate grammar and admissible input domain is accepted by the chosen terminating decision procedure. This needs the converse direction as well as soundness; it is stronger than the existence of a useful proof-producing checker.
3. **Negative realization.** A failing abstract witness corresponds to a particular admissible concrete input, and the semantic bridge transfers its failure. Realizing every abstract input is sufficient but stronger than the witness-specific condition needed for one refutation.

4. **Observation coverage.** The checked observations determine the property in question. Exhaustive finite enumeration needs a coverage theorem; an infinite separating family needs a theorem quantified over every member. Checking finitely many of its members is a different guarantee.

Representation	What positive evidence licenses	What is needed for a decision or refutation
Affine lengths of <code>List Nat</code>	Equal coefficients imply the universal length law through the interpreter theorem.	For the fixed grammar, zero and basis vectors determine coefficients and are realizable by lists. The supplied Lean model does not yet prove the coefficient criterion.
Affine lengths of <code>List Empty</code>	Equal coefficients are still sufficient.	Raw coefficient equality is incomplete for source equality: 0 and n agree at the sole realizable length 0. Positive length cannot refute a source law.
An enumerated finite domain	All enumerated cases plus coverage imply the universal claim.	Decidable cases and exhaustive coverage give a decision; rejecting a case requires a typed domain element.
Finite prefixes	A theorem for every prefix can imply sequence equality through joint separation.	One finite cutoff does not cover the infinite family; the quantifier over all prefixes is indispensable to this route.
A sample bank	Establishes the sampled instances.	A separate theorem relating those samples to the whole claim is needed for a universal conclusion.

Completeness therefore belongs to a tuple of source domain, admissibility, candidate grammar, specification, abstraction, and decision procedure. It is not a permanent badge on “length” or “finite evidence.” In particular, the parallel review’s `List Empty` row cannot be complete under its stated source-level interpretation. This revision checks a three-theorem Lean counterexample: the affine forms 0 and n agree on all `List Empty` inputs, their coefficient pairs differ, and the proposed converse implication is false. A procedure restricted to the realizable zero input can be complete; the unrestricted coefficient test is the part that fails.

6.4 Normalization is a construction

Obsidian’s distinction deserves explicit adoption. Dividing an integer triple by a common factor, permuting its entries, or selecting a prime exponent generally creates a new witness. A correct interface records the transformation and the properties preserved or established. It does not register the new triple as another property view of the old one.

The general workflow is therefore: refine the fixed input; construct a new witness under a theorem; refine that witness; transport only the claims licensed by the relation between the two. Definition by a specification can delegate the choice of a witness when the author has authorized that construction, but the relation and any uniqueness requirement remain part of the accepted target. This distinction matters as much for synthesized programs as for normalized mathematical counterexamples.

7 Worked case I: exact quotients without a vacuous benchmark

7.1 The theorem and the information it really needs

Several reports inspect integral and rational models with coefficients

$$a_2 = (b^p - 1 - a^p)/4, \quad a_4 = -a^p b^p / 16.$$

The integral expression uses integer division; the rational expression uses field division. Identical typography does not establish that casting one gives the other. The helper theorem needs exact divisibility.

Write F_0 for the following satisfiable parameter context:

$$a, b \in \mathbb{Z}, \quad p \in \mathbb{N}, \quad p \geq 4, \quad p \text{ odd}, \quad a \equiv 3 \pmod{4}, \quad 2 \mid b. \quad (1)$$

It is inhabited, for example, by $(a, b, p) = (3, 2, 5)$. From evenness and $p \geq 4$ we obtain $16 \mid b^p$, hence $4 \mid b^p$ and $16 \mid -a^p b^p$. Since a is congruent to -1 modulo four and p is odd, $a^p \equiv -1 \pmod{4}$. Therefore

$$4 \mid b^p - 1 - a^p, \quad 16 \mid -a^p b^p.$$

No Fermat equation, third coordinate, coprimality, or nonzero-coordinate assumption is needed for these two statements.

Proposition 7.1 (Exact quotient transport). *Let $n, d, q \in \mathbb{Z}$ satisfy $d \neq 0$ and $dq = n$. Under the canonical embedding $\iota : \mathbb{Z} \rightarrow \mathbb{Q}$,*

$$\iota(q) = \iota(n)/\iota(d).$$

Under an arbitrary unital ring homomorphism $j : \mathbb{Z} \rightarrow R$, the universally available conclusion is $j(d)j(q) = j(n)$. An inverse formula additionally needs $j(d)$ to be a unit.

Proof. A ring homomorphism preserves the balance equation. In $\mathbb{Q}$, the embedding preserves nonzeroness of d , so its image can be inverted. In a general ring, the image need not be invertible; the balance equation alone is what follows. $\square$

The interface separates three facts: exact division in the source, preservation under a homomorphism, and inversion in the target. Even a nonzero integer can map to zero in positive characteristic. A regular element can be cancelled from an equality without being a unit: 2 in $\mathbb{Z}$ is the elementary example. The requirement solver must select the theorem matching the requested use.

7.2 A reusable checked arithmetic package

The parallel review contributes `FreyExact.lean`; this revision compiles an unchanged copy against Lean 4.32.0 and the installed Mathlib dependency snapshot [12, Section 7.3]. Its six named theorems prove two arithmetic helpers, the two divisibility claims, and the two casts to $\mathbb{Q}$. Seven examples check the coefficient values -53 and -486 , four removed- premise neighbors, and the inexact cast of $1/4$. A separate positive probe also checks the conjunction of the assumptions for $(3, 2, 5)$.

The theorem interfaces improve the proposed property package. For a_4 , only $2 \mid b$ and $4 \leq p$ are needed; oddness of p and $a \equiv 3 \pmod{4}$ are used for a_2 . An operation requesting only a_4 should not inherit the whole shared F_0 context merely because both coefficients belong to one larger construction. Conversely, these sufficient premise sets are not a claim of globally weakest preconditions.

The quotient proofs call `Int.cast_div`. Its destination condition is nonzeroness of the *cast denominator*, a distinction that matters outside $\mathbb{Q}$. The result establishes these coefficient equations. It does not yet prove equality of two complete curve models or implement the property-aware use site. The four printed interface-theorem inventories use only the standard axioms `propext`, `Quot.sound`, and, in three cases, `Classical.choice`; no native-evaluation axiom is involved.

7.3 A proposed authoring trace

Write F_0 for the assumptions in (1). The following is illustrative source notation, not an implemented language:

```
let a, b : Int and p : Nat satisfying F0
let q2 := exact quotient of (b^p - 1 - a^p) by 4
let q4 := exact quotient of -(a^p) * b^p by 16
define integral_model using q2 and q4
obtain rational_model by base change to Rat
show its displayed coefficients by exact-quotient transport
```

The elaborator owes divisibility proofs, the balance equations defining the chosen quotients, and coefficientwise equality with the intended rational model. It may hide routine proof terms after acceptance, but the change of coefficient domain remains visible. If migrating an existing development with two independently defined models, retain a compatibility theorem rather than silently replacing the old definition. Fiber and Obsidian develop this example particularly well [2, Section 9][6, Section 6].

7.4 Why the surrounding contradiction matters

A full counterexample package in a proof of Fermat’s Last Theorem is eventually shown inconsistent. From an available contradiction, every helper property follows. Such a proof can be completely kernel-correct while failing to test exact-division inference or to follow the advertised arithmetic argument.

The solution is not to prohibit inconsistent contexts: proof by contradiction is ordinary mathematics. Extract the arithmetic helper under (1), prove the application by a short adapter, and evaluate the helper with a specified support and method policy. When a named route is required, a contradiction shortcut should fail the *method* test even though its theorem is true in the supplied context. A syntax check of which hypothesis names occur is insufficient; dependencies can be hidden inside a called theorem. The accepted derivation needs inspectable transitive support.

The paired neighbors isolate the requirements:

- (3, 2, 4) violates oddness but keeps the lower bound; the first numerator is not divisible by four.
- (3, 3, 5) violates evenness of b ; again the first divisibility fails.
- (3, 2, 3) violates the exponent lower bound while retaining oddness; the second numerator is not divisible by sixteen.

These are finite counterexamples to weakened helper statements, not counterexamples to the original source theorem. A proposed “missing oddness” test using $p = 2$ also violates the lower bound and cannot isolate the cause; Fiber’s $p = 4$ neighbor is the better choice. Satisfiable support and

single-premise mutations should be explicit requirements for the next benchmark.

8 Worked case II: finite observations of infinite operators

8.1 The behavioral hypothesis that makes restriction lawful

Let R be a unital commutative ring and $\mathcal{S} = R^{\mathbb{N}}$ the module of streams. For $N \in \mathbb{N}$, write $\pi_N : \mathcal{S} \rightarrow R^N$ for the first N coordinates and $\iota_N : R^N \rightarrow \mathcal{S}$ for extension by zero. Then $\pi_N \iota_N = \text{id}$.

Definition 8.1. A map $F : \mathcal{S} \rightarrow \mathcal{S}$ is *prefix-local* when

$$\pi_N(s) = \pi_N(t) \implies \pi_N(F(s)) = \pi_N(F(t))$$

for every N, s, t .

This is a relational property of the operation. Its output type alone says nothing about locality. For a prefix-local map, define $F_N = \pi_N F \iota_N$. Since s and $\iota_N \pi_N(s)$ have the same prefix,

$$\pi_N F = F_N \pi_N. \quad (2)$$

If F is linear, so is F_N . If F, G are prefix-local, their composition is prefix-local and $(FG)_N = F_N G_N$, by applying (2) twice. Zero extension is an implementation choice whose irrelevance to the observed result has a theorem; it need not become an author-visible arbitrary choice.

Theorem 8.2 (Reverse a one-sided inverse through every finite prefix). *If $F, G : \mathcal{S} \rightarrow \mathcal{S}$ are linear and prefix-local and $F \circ G = \text{id}$, then $G \circ F = \text{id}$.*

Proof. At each N , the restriction equations give $F_N G_N = I$ on R^N . Finite square matrices over a commutative ring are directly finite: $AB = I$ implies $BA = I$. To see this for $N > 0$, determinants give $\det(A) \det(B) = 1$, so $\det(A)$ is a unit and the adjugate constructs a two-sided inverse C of A . Then $B = (CA)B = C(AB) = C$. The zero-dimensional case is immediate.

Consequently $G_N F_N = I$, and every prefix of $G(F(s))$ equals the same prefix of s . To obtain equality at coordinate k , take $N = k + 1$. Function extensionality yields $G(F(s)) = s$. $\square$

Gneiss and Tephra use this as a principled generalization of finite triangular matrix arguments [3, 8]. It makes observation preservation, finite algebra, and coverage by all observations separate obligations. Checking $N = 1, \dots, 1000$ does not establish the quantified theorem.

8.2 Negative neighbors reveal the exact scope

The left and right shifts on streams satisfy

$$(Ls)(n) = s(n+1), \quad (Rs)(0) = 0, \quad (Rs)(n+1) = s(n).$$

Both are linear and $LR = \text{id}$, but RL erases coordinate zero. The missing premise is prefix locality of L , not linearity or the ability to form a finite vector. Product-topology continuity would not repair this example.

Conversely, on integer streams the coordinatewise doubling and integer-halving maps are prefix-local and form a one-sided inverse pair, but integer halving is not $\mathbb{Z}$ -linear and the reverse composition need not be the identity. Thus the two hypotheses have distinct jobs.

For values in an arbitrary additive commutative group M , do not pretend M^N is finite free over $\mathbb{Z}$. One valid route proves an integer triangular kernel identity first and then applies it through the integer scalar action on M . Preserving this route matters for keeping a library theorem’s original generality.

8.3 A reusable interface theorem added by this synthesis

The argument suggests a generic interface independent of streams. Suppose observations $o_i : A \rightarrow O_i$ are surjective and jointly separate points; $f, g : A \rightarrow A$ commute with operations f_i, g_i on the observations; and each observation space supplies

$$(\forall u, f_i(g_i(u)) = u) \implies (\forall u, g_i(f_i(u)) = u).$$

Then $f \circ g = \text{id}$ implies $g \circ f = \text{id}$. Surjectivity lifts the global left-inverse equation to every observed input; local reversal gives the right-inverse equation; separation reflects it to the original object.

The two uses of adequacy are different: surjectivity covers the local domain, and joint separation recovers exact global equality. This generic implication is newly checked here in `reverse_via_observations`. The local reversal law is an explicit hypothesis of that Lean theorem; the finite-matrix proof above remains a written proof in this report. This is a reusable interface factorization, not a new theorem of linear algebra claimed as historically novel.

9 Worked case III: analytical contracts and strength of conclusions

9.1 Orbital comparison without unnecessary hypotheses

Schist extracts a reusable argument from probability-law uniqueness. Let X be a topological space, $T : X \rightarrow X$, and $p \in X$. Assume $T^n(x) \rightarrow p$ for every x . Let $f, g : X \rightarrow \mathbb{C}$ be continuous at p and satisfy $f(p) = g(p)$. Suppose

$$f(x) = a(x)f(Tx), \quad g(x) = a(x)g(Tx), \quad |a(x)| \leq 1.$$

Then $f = g$: subtracting and taking norms gives

$$0 \leq |f(x) - g(x)| \leq |f(T^n x) - g(T^n x)| \rightarrow 0.$$

The result needs neither global continuity of f, g nor continuity of T once orbit convergence is provided. The multiplier may vanish; nothing is cancelled.

For characteristic functions of Borel probability measures on $\mathbb{R}$, $T(t) = qt$ with $q \in \mathbb{R}$ and $|q| < 1$, continuity at zero, the shared value one, and the recurrence supply this interface. Equality of characteristic functions then gives equality of the original measures by its separate uniqueness theorem. This proves at most one law with the prescribed recurrence; it does not construct a law, establish moments, or infer that arbitrary functions with this profile are characteristic functions [7].

A broader sufficient route assumes only that the orbit products $A_n(x) = \prod_{j < n} a(T^j x)$ are bounded by some finite M_x . Iterating the recurrence yields $|f(x) - g(x)| \leq M_x |f(T^n x) - g(T^n x)| \rightarrow 0$. The pointwise bound gives $M_x = 1$, but is not necessary. A registry can offer both routes. There may be several incomparable sufficient contracts, with no unique “weakest registered guarantee.”

9.2 Local equality, null boundaries, and quantitative observations

Three other examples prevent “regularity” from becoming an imprecise label.

1. **Differentiating an identity.** An identity valid only on a region must be available in a neighborhood of the differentiation point. Agreement at the point alone is insufficient. Iterated differentiation must preserve the local hypotheses at every step.
2. **A symmetric distribution function.** Let X be a measurable real-valued random variable and $F(x) = \Pr[X \leq x]$. If its law is invariant under $X \mapsto 1 - X$, then

$$F(1 - x) = \Pr[X \geq x] = 1 - F(x) + \Pr[X = x].$$

The familiar $F(1 - x) = 1 - F(x)$ needs no atom at x . A point mass at $1/2$ is symmetric and violates that formula at $x = 1/2$. Basalt’s null-boundary obligation is substantive mathematics, not annotation overhead [1].

3. **A coupling bound.** For measurable real-valued random variables X, Y on a common probability space, $t \in \mathbb{R}$, and $\varepsilon \geq 0$, if $|X - Y| \leq \varepsilon$ almost surely, then

$$|\mathbb{E}e^{itX} - \mathbb{E}e^{itY}| \leq \min(2, |t|\varepsilon).$$

Use $|e^{iu} - e^{iv}| \leq \min(2, |u - v|)$ and the expectation bound. This quantitative characteristic-function statement is not by itself a uniform bound on distribution functions. Fiber’s example benefits from naming the requested observation precisely [2].

9.3 Summability licenses an interchange

Obsidian’s Mellin-family study emphasizes the difference between pointwise convergence and a summability guarantee that justifies integration term by term. For integrable h_n on a measure space, a hypothesis such as

$$\sum_n \int |h_n| d\mu < \infty$$

supplies an L^1 route to convergence and interchange. It carries a measure, integrability evidence, and a norm-summability assertion. They cannot be replaced by the statement that $h_n(x) \rightarrow 0$ or that partial sums converge pointwise [6].

For a basic failure, on $(0, 1)$ let $f_n(x) = n \mathbf{1}_{(0, 1/n)}(x)$ for $n \geq 1$. Then $f_n(x) \rightarrow 0$ at every $x > 0$, while $\int_0^1 f_n = 1$. A language may make the domination or L^1 argument concise, but it cannot infer an interchange from the shape of a limit expression. This is a useful nonalgebraic transfer task after exact division.

9.4 Upper bounds are not optimality claims

Trellis carefully separates a Lipschitz bound from the assertion that its constant is least. On a real interval, a derivative bound can yield a Lipschitz constant by the mean value theorem. If the derivative attains magnitude L at an interior point, any Lipschitz constant K must satisfy $L \leq K$ by taking limits of difference quotients there. The attainment argument adds the missing optimality direction. A checker that merely proves $|f'| \leq L$ has not proved that L is sharp [9].

This pattern generalizes: a witness is not a complete solution set; a bound is not a least bound; a preservation theorem is not a reflection theorem. Source syntax should make the demanded strength explicit before selecting a method.

10 Relational interfaces and a correction to the corpus

10.1 Contracts belong to the whole configuration

Basalt’s exactness and completion examples, Moraine’s naturality examples, and Tephra’s invariant-subtype construction show why a collection of isolated properties is not enough. A relation may mention several maps, their domains, the connecting arrows in a diagram, or a chosen action.

For commutative rings R, S , an ideal-preserving ring map $f : R \rightarrow S$ with $f(I) \subseteq J$ induces maps $R/I^n \rightarrow S/J^n$. To obtain a map on the inverse limits, these maps must commute with the transition maps. A collection of unrelated levelwise maps, or even levelwise isomorphisms, does not establish the coherent tower map. Compatibility is a proof obligation about the family [1].

For an action on A and an invariant predicate P , the action can be restricted to $\{x : A \mid P(x)\}$. The action laws follow from the original laws and subtype equality. The implementation must retain the same action and prove invariance; the existence of a similarly named action is irrelevant. Such adapters are plausible synthesis targets because their data assembly and proof assembly are both structural [8].

10.2 Finite indexing does not imply finite-dimensional factors

Moraine’s finite tensor-family reconstruction contains a premise omission in its exposition. It introduces a finite family of commutative A -algebras H_i and asks for a representing algebra W that is finite and free over A . To retain those conclusions, it must also assume that *each H_i is finite and free as an A -module*. The pinned original Lean theorem includes exactly these two assumptions; they are missing from the reconstructed setup [5, Section 6.2][13].

The universal-property component has the form of natural equivalences

$$\text{AlgHom}_A(W, D) \simeq \prod_i \text{AlgHom}_A(H_i, D).$$

For a singleton family, natural representability forces W to be isomorphic to its single factor. Taking $A = \mathbb{Q}$ and $H = \mathbb{Q}[X]$ gives a factor of infinite $\mathbb{Q}$ -dimension. A finite index set does not make the representing algebra finite dimensional. Universal representability and naturality can remain valid while the extra finite/free conclusion fails.

The correction is to restore the factor hypotheses when requesting finite/free output, or to weaken the output guarantee when those hypotheses are absent. This is an error in the report’s presentation, not a flaw identified in the original Lean source. The source is retained as pinned read-only evidence; its theorem was not recompiled here.

The example should become a deliberate test of the proposed authoring layer: remove a factor’s module-finiteness assumption and ensure that the finite conclusion becomes an open obligation. A surface designed to preserve premises must be evaluated on exactly this kind of plausible omission.

11 Leant: from candidate identity to proved behavior

11.1 The current boundaries deserve an accurate baseline

The reports reviewed a much richer Leant synthesis pipeline than bare type inhabitation. A bounded fresh source review for this synthesis uses the immutable commit 823259f (the complete identity is in the reference); local edits in the external checkout are preserved and are not attributed to that commit. The findings are source observations, not a new Leant executable or regression test run [14].

The source separates several meanings of acceptance:

- **Verified** records acceptance by a supplied verification callback. Its nominal, opaque wrapper preserves association; it is not automatically a kernel proof, solver certificate, or behavioral theorem. The combined behavioral-verification entry point requires successful type and behavioral callbacks before admitting a candidate, so this boundary is stronger than type checking alone while still lacking a retained proof field.
- The behavioral decision path generates an exact proposition containing the candidate and uses **decide**. A failed positive attempt is not refutation; a successful check of the negation is needed. What is established depends on the actual proposition supplied. It may be finite or universal; the code does not turn a finite example into a universal statement.
- Behavioral selection seals occurrence membership, total partition, and association with the original candidate order. Its domain-specific evidence policy must supply the authority for a behavioral decision.
- Length contracts supply explicit provider laws and argument roles. The adapter binds them to retained identities and a specific translated candidate. These laws remain supplied assumptions unless a separate proof links them to the providers' Lean semantics.

The adapter's explicit warning that abstract replay does not by itself authorize a source-level behavioral refutation is consistent with the round-3 proposals. The realizability example below is a requirement for a stronger future bridge, not evidence of an already established Leant bug.

11.2 Why the bare function type is too weak

For a sorting request, $\text{List}(A) \rightarrow \text{List}(A)$ permits constant empty output. Length preservation excludes that candidate for inhabited nonempty inputs but still permits a list filled with one fixed element. Sortedness permits that filled list as well. The intended contract needs both sortedness and permutation of the *actual input*, universally quantified over inputs. Neither plausible names nor a few successful evaluations supply that theorem.

A useful synthesis request contains the carrier type, fixed structures, input guard, relational post-condition, permitted providers, and source context. Search may first build a typed candidate and then discharge its behavioral obligations, or use contracts to constrain partial candidates earlier. In either case, acceptance must ultimately link the exact rendered Lean term to the original specification. Search order, costs, and heuristic ranking can change without changing this boundary.

11.3 A concrete abstract interpretation: list lengths

For a small list-expression grammar with inputs, empty list, cons, append, reverse, and a fixed element map, define concrete evaluation and a length model. The model sends empty to 0, cons to addition of 1, append to addition, and reverse/map to their argument's length. Structural induction gives

$$|\text{eval}(e, \rho)| = \text{model}(e, i \mapsto |\rho_i|). \quad (3)$$

Moraine's supplied Lean file now proves this theorem for its fixed **List Nat** grammar, together with promotion of a *supplied universal* model law to a law about actual lists.

For this grammar, the normalized arithmetic model is affine: $c + \sum_i a_i n_i$ with natural coefficients. Equality of two such expressions for all $n \in \mathbb{N}^k$ is equivalent to equality of constants and coefficients. Evaluate at 0 to identify constants and at each basis vector to identify coefficients; conversely matching coefficients give equality at every input. Thus zero and basis vectors yield a complete argument for this fixed affine fragment, not a general justification for finite behavioral testing.

The Python model implements the normalization and comparisons. The supplied Lean specimen proves (3), but does not formalize the coefficient normalizer or establish that arbitrary rendered Leant candidates are represented by this grammar. Those are explicit next proof obligations.

11.4 Positive evidence and negative evidence need different bridges

Let $o_A : A \rightarrow U$, $o_B : B \rightarrow V$, $f : A \rightarrow B$, and $\hat{f} : U \rightarrow V$ satisfy

$$o_B(f(x)) = \hat{f}(o_A(x))$$

on admissible inputs. A positive abstract theorem over *every* $u \in U$ also holds at each realized $o_A(x)$. If the abstract property implies the requested concrete property, this yields a positive proof. Quantifying over extra unrealizable abstract inputs can make the proof harder, but does not make the positive implication unsound.

A negative abstract witness u is different. It refutes the concrete universal specification only when an admissible x realizing u is available and a true concrete specification would imply the abstract one. Surjectivity of the input observation restricted to admissible inputs is one sufficient condition, but a realization proof for the particular witness is enough. This synthesis's **refute_via_realization** checks that generic implication with the admissibility, transfer, and realization premises explicit.

For list length, the exact image is

$$\{n \in \mathbb{N} : \exists xs : \text{List}(A), |xs| = n\} = \{n : n = 0 \vee A \text{ is inhabited}\}.$$

At length zero use the empty list. A positive-length list supplies an element; conversely an element can be repeated any number of times. Consequently **dropAll** : **List Empty** $\rightarrow$ **List Empty**, defined by constant empty output, universally preserves length. The abstract assignment $n = 1$ is not a counterexample to this program: there is no corresponding input. Karst's unchanged Lean specimen now proves this positive counterexample to an *incorrect rejection rule* [4, 8, 1].

For a product of observations, realizability can also impose correlations. Two individually possible abstract values may not be jointly obtainable from the same input. A pair of independent scalar certificates cannot silently replace the relation required by a coupled-output request. The proposed bridge must index the entire input and output configuration, including guards and dependencies. Leant already has joint two-output spine contracts at the reviewed snapshot. The next semantic bridge should use that relational interface rather than reducing it to unrelated scalar checks.

11.5 Polymorphism does not supply parametricity for free

A term with a polymorphic type has passed typing, not an independently stated parametricity theorem. Classical Lean can distinguish properties of types using classical reasoning, so syntactic polymorphism alone should not authorize a uniform transfer law over arbitrary types. Require either a proved provider law or a restricted grammar with an adequate semantic theorem. The official Lean axioms documentation explicitly discusses the failure of unrestricted parametricity in the presence of classical choice [17].

This distinction protects a useful optimization. A syntactically restricted fragment may enjoy uniformity strong enough to prune candidates by an inhabited test type, while arbitrary user definitions do not automatically belong to that fragment. The fragment boundary and its proof are the feature; the convenient polymorphic spelling is not the proof.

11.6 The synthesis experiment that would matter

Evaluate Leant on actual obligations generated by the proposed interface: assembling invariant-subtype maps, composing behavioral contracts with a dependent intermediate, building a quotient-transport adapter, or combining two length-preserving operations. Include direct theorem-lookup controls so search is not credited for work a single existing lemma already performs.

Record typed candidates, rejected behavioral candidates, proof-producing successes, unsupported fragments, budget exhaustion, and proven refutations separately. A failed search is not nonexistence of an implementation. Replay the exact suggested source in the original proof state, including all goals, and retain the resulting proof. A cached search result is useful only when its target, contexts, provider laws, and acceptance policy still match or are connected by checked adapters.

A *sketch* is a partial candidate with designated holes and a specified set of permitted completions. A counterexample to one completed program excludes that program. Rejecting the entire sketch requires a proof that every permitted completion violates the contract, or an explicitly heuristic status that carries no such logical conclusion. Backward requirements can prune sketches soundly only when their necessary-condition theorem covers all of those completions.

12 Certified computation: what is now actually checked

12.1 The general proof chain

For a reflective certificate service, separate at least these steps:

1. Define finite expression and certificate data with explicit coefficient domains, dimensions, and interpretation.
2. Prove that its arithmetic operations preserve interpretation.
3. Prove that acceptance by the checker implies the interpreted relation.
4. Connect the reified inputs and outputs to the original Lean expressions.
5. Establish the actual checker execution under the chosen trust policy.
6. Apply the resulting theorem to the exact original target and its guards.

For ideal certificates over a commutative ring, a candidate identity $p = \sum_i q_i g_i$ yields $p(x) = 0$ at valuations where every $g_i(x) = 0$. The coefficient-list or sparse-map equality must first be related to polynomial evaluation. A Boolean accepting its own unproved representation does not by itself establish that mathematical implication. Many round-3 reports now give explicit written denotation arguments, a substantial improvement over merely writing the type of a hypothetical soundness theorem.

12.2 Schist closes one deliberately small chain

Schist’s reference language has natural-number literals, one variable, addition, and multiplication by natural constants. Its normal form is a pair (a, b) denoting $ax + b$. The supplied Lean file proves

$$\text{eval}(e, x) = a_e x + b_e,$$

defines equality checking by equality of normal forms, proves the checker sound, and uses it to prove the actual target

$$3(x + 2) = 3x + 6 \quad (x \in \mathbb{N}).$$

It also checks that replacing 6 by 7 is rejected. All these declarations compile unchanged in the fresh run. The axiom inventory for the denotation, soundness, and original-target proofs contains only `propext`; the corruption rejection has no axioms.

This is a complete narrow example, including definitional reconstruction of its hand-authored source expressions. It is not a multivariate ideal checker, an arbitrary-source reifier, or proof that a frontend never reifies the wrong target. It also computes its own normal forms, so it does not establish the performance of a large external certificate-discovery service. Its value is that the smallest intended proof chain now has actual kernel evidence.

12.3 Moraine supplies a different, complementary link

The checked list interpreter theorem establishes a semantic abstraction of a fixed program grammar. It does not use a polynomial checker, and its promotion theorem takes the universal model property as a hypothesis. This should become the first behavioral acceptance bridge, with normalization and rendered-candidate correspondence proved next. Fiber’s generic `certificate_bridge`, by contrast, assumes both checker soundness and execution; successful compilation validates the composition lemma, not those assumptions. These distinctions prevent theorem counts from becoming a false measure of implementation completeness.

12.4 Existing tools and execution policy remain controls

The current mathlib `fun_prop` tactic already proves registered function properties compositionally and can use local assumptions and side-condition dischargers. Lean’s `mvngen` already generates verification conditions from Hoare-style specifications using registered specification lemmas. These are relevant baseline capabilities, although their targets and interfaces are not identical to every proposed mathematical refinement. At the inspected Lean 4.32.0 pin, `mvngen` explicitly warns that it is experimental [15, 16, 12].

The peer’s `Supply3.lean` is a mixed probe: its final, deliberately misapplied `mvngen` example errors, as its appendix discloses. It is not a successful whole-file compilation. This revision’s separate positive probe removes that misuse and checks `fun_prop`, the composite from equality on an open neighborhood

to derivative equality, and finite-matrix inverse reversal. Derivative equality here concerns Lean’s totalized derivative; it does not establish differentiability. The probe does not test an effectful verification workflow or the concrete analytical interchange examples.

The matrix probe uses `mul_eq_one_comm`, replacing the deprecated `Matrix.mul_eq_one_comm`. Its general interface is the reversal law in an `IsDedekindFiniteMonoid`. To generalize Section 8 beyond commutative coefficient rings, the required instances are for *each finite matrix monoid*. A reversal property of the scalar ring alone should not be silently promoted to this stable-finiteness condition.

Likewise, `grobner`, `linear_combination`, arithmetic tactics, and ordinary theorem search belong in the comparison for the supported fragment. A bounded tactic failure does not prove the goal false, and reusable external certificate export should not be assumed without checking the actual API.

Native evaluation is a separate execution route with a version-sensitive axiom policy. The earlier microbenchmark does not establish a universal crossover threshold, and the new Python tests do not measure kernel checking. Start with the direct route on the actual checker; measure interpretation, reification, certificate checking, startup, and proof retention separately before deciding whether an accelerated route earns its additional assumptions. No new native evaluation axiom is used by the fresh checks in this report.

13 An implementation that tests the authoring hypothesis

13.1 Reuse Lean’s state; add an interface to its evidence

The minimum additional state is an index from elaborated propositions to accepted proof terms, a bounded rule profile, source-to-obligation links, and records needed to reproduce accepted generated work. Lean already owns local variables, assumptions, instances, definitions, proof terms, and elaboration snapshots. The new layer must justify its additional state by operations it enables, such as explaining an unresolved requirement or replaying a generated adapter after a source change.

Need	Existing mechanism to retain	Additional feature to evaluate
Known local facts	Hypotheses, <code>have</code> , bundled projections	Demand-indexed retrieval and explanations.
Function properties	Theorem attributes and <code>fun_prop</code>	A coherent use-site interface across several relation kinds.
Behavior specifications	Dependent function types and contract theorems	Requirement propagation and proof-linked candidate acceptance.
Scopes and edits	Lean contexts and elaboration snapshots	Dependency slices, exact-source replay, and migration records.
Mathematical objects	Definitions and chosen structure arguments	Notices when interpretation changes; explicit construction/view distinction.
Computation	Existing tactics and checked definitions	Certificate interfaces and retained target reconstruction when needed.

13.2 Evidence after rewriting and incremental elaboration

The parallel review identifies two concrete gaps: an expression may be rewritten before its property row is consulted, and Lean may reconstruct its local context after an edit [12, Sections 8.3 and 11]. These deserve acceptance tests before an editor-wide evidence index is attempted.

Suppose $p : P(t)$ and simplification replaces t by u . If t and u are definitionally equal in the retained context, ordinary Lean type checking can reuse the proof. Otherwise, a checked equality $e : t = u$ permits the explicitly typed transport $e \triangleright p : P(u)$. A relation weaker than equality needs its own consumer theorem. A normal-form string or matching pretty-printed name licenses none of these operations. Rewriting only the consumer’s goal also need not delete the original fact: the index may retain the old anchor and apply an explicit bridge at use time. Proposition rewriting is another common case: from $p : P(t)$ and a checked $h : P(t) \leftrightarrow Q(u)$, the forward implication gives $Q(u)$ without asserting $t = u$. The retained adapter must match the propositions actually proved and requested.

For dependent indices the target predicate itself may need transport. The system must preserve universe instantiations, dependent arguments, and chosen structure dictionaries, then ask Lean to check the transported proposition. A first prototype should restrict supported transport shapes and expose the remaining obligation rather than silently approximate this correspondence.

Give index entries the lifetime of an elaboration snapshot. After a declaration is rebuilt, reconstruct the index from the new context, or replay entries only through a checked context embedding that maps the actual local declarations and their dependencies. Old local identifiers and receipt hashes are routing information; they are not a context embedding. Initially rebuilding may be cheaper and easier to justify than migrating the cache. Measure both before making persistence a semantic feature.

The first stress cases should simplify an anchor, change a dependent index, rename a binder without changing meaning, replace a hidden structure argument, and rebuild a declaration after an upstream edit. Record transported facts, invalidated facts, reconstruction cost, and whether the displayed explanation still describes the accepted proof. This tests an implementation invariant that the small conservative-translation theorem alone does not establish.

13.3 Freeze meaning while allowing proof work to continue

Before accepting proof search, fix the proposition and its meaning-bearing arguments: objects, structures, domains, measures, quantifier order, branch, precision, and result strength. This does not require solving every proof metavariable in advance. Proof holes can remain goals; a requested witness can remain a construction problem when its specification authorizes the choice.

The dangerous case is letting search instantiate an unresolved statement parameter merely to make the resulting proposition easy. An accepted candidate should be checked against the fixed target or against an explicitly authorized construction specification. This policy should operate on elaborated terms, not a blacklist of surface words or a rule banning all metavariables.

13.4 Four small exit gates

1. **Retain the checked foundations.** Reproduce the seven small source files and keep the tiny affine checker and list interpreter as controls. Add the missing coefficient-normalization theorem only for the chosen list DSL. Do not infer a link from Python or Leant merely because the mathematical formulas resemble the Lean definitions.
2. **Implement one property-aware use site.** A binder introduces fixed objects with explicit proof obligations; a registered operation requests those proofs; a bounded engine discharges them or shows the frontier. The arithmetic task is exact quotient transport under (1), using ordinary divisibility, cast, and coefficient-extensionality lemmas, starting from the checked Frey helpers in Section 7.2. Schist’s natural-number checker is a boundary control, not an integer-division solver.

3. **Close one behavioral loop.** For the fixed list grammar, connect the exact candidate syntax to evaluation, prove the model normalization, transfer accepted universal specifications, and reconstruct concrete negative witnesses when permitted. Integrate one actual Leant producer only after its provider laws and rendering link have a checked meaning.
4. **Run a transfer and repair study.** Apply the same interface to a fresh nonalgebraic contract, edit a hypothesis or provider law, and measure whether both proof reconstruction and explanation remain useful. At this point decide whether a separate narrative syntax is warranted.

These are exit conditions, not calendar estimates. The small checker and the behavioral interpreter answer different questions; completing one does not clear the other's gate. A general multivariate checker, rich definition-package language, or editor-wide persistent workspace can wait until a real workload requires it.

The peer proposes extending Schist's checker to Trellis's rational affine nonnegativity certificates and a reifier for real expressions [12, Section 9.4]. This is a useful alternative second checker, but it adds ordered semantics, a nonnegativity theorem, and a new expression bridge. The existing natural-number equality proof supplies an acceptance pattern, not those results. Compare that route with directly using established Lean arithmetic tactics before claiming it is the smallest implementation.

13.5 Three acceptance checks, with separate failure states

Theorem truth asks whether the retained Lean term proves its stated proposition under the selected axiom policy. **Statement fidelity** asks whether that proposition matches the authored request. **Method fidelity** asks whether the accepted argument satisfies any explicit requirement about which mathematical route or support may be used.

A compact proof by a terminal contradiction can pass the first two and fail the third. A proof of a weakened relation can pass kernel checking while failing the second. A correct certificate about a different reified expression can fail at target reconstruction. These should be separate diagnostic states, not a single red/green status with an unexplained rejection.

Three dependency questions should remain separate: which premises a theorem's public interface requires, which premises this application supplies, and which dependencies occur in the retained proof. None alone establishes that a premise is mathematically indispensable. A decorative occurrence of a theorem constant does not establish that its advertised method justifies the step.

Named reasons should therefore have two explicit modes: a search hint and a required method. The required mode may be checked using a small accepted derivation or a specified rule set; the project must define what that check guarantees. It should not claim to infer a human author's intention from any arbitrary proof term.

13.6 When a kernel change would be justified

None of the worked contracts requires new logical expressiveness. Possible kernel research includes native refinement constructs or carefully restricted conversion extensions, but arbitrary semantic subsumption would make equality checking depend on unrestricted theorem proving. Adding it prematurely also creates library, trust, tooling, and migration costs.

A serious comparison should isolate the alleged obstruction: repeated transport terms, inability to express a useful compositional elaboration rule, or unacceptable checking overhead for the conservative

encoding. Prototype the smallest alternative and compare it against the best Lean extension. Do not infer a foundational deficiency from a source file whose bulk is generated options, imports, or packaging that a library can already remove.

14 Evaluation that can change the design decision

14.1 Separate the causes of improvement

The following five treatments combine the clearest proposals from Fiber, Schist, Obsidian, and Trellis. They need not all be fully productized before the first pilot.

Treatment	Available facilities	What the comparison isolates
L0	Current ordinary Lean on the task	Initial authoring and repair baseline.
L1	Lean plus the new mathematical packages	Benefit of library abstraction.
L2	L1 plus the same search and certificate services	Benefit of automation and computation.
L3	L2 plus property-aware binders, requirements, and diagnostics	Incremental value of the elaboration interface.
L4	L3 plus a mathematical narrative surface/document view	Incremental value of language and presentation.

The comparison that justifies a new language is not L0 versus L4 alone. The new mathematical packages and applicable theorem registrations should be available to L1–L4; the same added search and certificate services should be available to L2–L4. Existing automation such as `fun_prop` and applicable `mvngen` support belongs in the ordinary Lean baseline as well. Record both first-use infrastructure cost and its amortization across subsequent tasks. An adapter paid for by the research team still counts.

Before implementing new input syntax, add the peer’s renderer-only control: present the same accepted L2 proofs through a read-only mathematical view, with their statements and proof services unchanged [6, 12]. This separates a comprehension gain from display from a gain due to property inference or source notation. Such a control can run alongside the L2–L3 comparison; a successful renderer does not by itself justify a new parser.

14.2 A workload portfolio with different failure modes

Use the following as development tasks, then recruit genuinely unseen transfer tasks. Any file already inspected or used to design the interface has ceased to be held out.

- **Exact quotients:** source semantics, helper support, and target casts.
- **Length contracts:** syntax/semantics linkage, provider laws, and realizable refutation.
- **Analytic contracts:** a local derivative, an L^1 interchange, or orbital comparison, testing whether the interface generalizes beyond algebra.
- **Relational maps:** a coherent observation family or invariant-subtype adapter, with multiple objects and dependent premises.
- **Already-short Lean:** a terminal theorem application, serving as a control against making concise proofs longer or more fragile.

Measure active author time, explicit routine obligations, time to diagnose a failure, semantic-reading error, edit repair effort, search latency, proof size, checking cost, and retained evidence size. Report distributions and paired differences, not only the most attractive example or an aggregate source-line reduction. The current tests establish none of those productivity metrics.

Add the peer review’s per-declaration telemetry: index entries and bytes, registered-rule firings and retries, transport-proof size, rebuild time, and the fraction of obligations solved by one existing lemma versus composition [12, Section 11]. Compare these with ordinary **exact?** and applicable property automation under the same registrations. A rule that never fires or only duplicates cheap lookup is a candidate for removal. Report the registration and maintenance effort as well as the cost of a successful run.

14.3 A compact adversarial acceptance suite

The next implementation should execute paired positive and negative cases at the real boundary. The table is a prioritized specification, not a claim that the language already passes it.

Case	Valid neighbor	Failure that must be identified
Identity	Reuse proof at the same elaborated object.	Same printed name but different object or structure.
Rewrite	Reuse by conversion or a checked transport to the rewritten proposition.	Match a normalized string while losing dependent or structure arguments.
Rebuild	Reconstruct the index or check a context embedding after an edit.	Reuse stale local identifiers as if the context had not changed.
Scope	Typed weakening into a child context.	Reuse a sibling branch’s nonzero hypothesis.
Meaning	Leave a proof hole for a fixed target.	Solve a data hole by changing the requested proposition.
Composition	Prove the next operation’s precondition from the actual intermediate.	Compose arrow types while omitting the behavioral bridge.
Cycles	Derive a guard from an independent fact.	Use a conditional conclusion to establish its own guard.
Quotients	Use the divisibility evidence and destination unit.	Confuse integer division, balance transport, and field inversion.
Support	Prove the helper under the satisfiable arithmetic assumptions.	Advertise exact-division reasoning while using a disallowed contradiction shortcut.
Tensor	Finite/free factors yield the corresponding module guarantee.	Infer finite dimension from finite indexing alone.
Locality	A prefix-local linear map yields coherent finite sections.	Use a left shift or nonlinear coordinate map in the reversal theorem.
Realization	Produce an admissible list witnessing a failing abstract length.	Reject List Empty behavior using abstract length one.
Correlation	Realize a pair of observations from one input.	Combine incompatible scalar witnesses.
Checker	Normal forms agree and reification matches the original target.	Accept a certificate for an altered target or corrupted expression.
Strength	Bound plus attainment establishes a least constant.	Publish sharpness after proving only an upper bound.
Interchange	Supply domination or an appropriate L^1 theorem.	Exchange limit and integral from pointwise convergence alone.
Replay	Recheck the exact selected source and all proof goals.	Present success after checking a different candidate or an incomplete state.

Case	Valid neighbor	Failure that must be identified
Migration	Create a new receipt after rechecking under the new policy.	Relabel historical evidence after a structure or axiom-policy change.
Publication	Check each displayed assertion, including unused ones.	Accept the final theorem while publishing an unrelated false intermediate claim.
Domain	Keep a function on a refined domain, or construct a justified extension.	Extend a function on $\{x : A \mid P(x)\}$ to all of A without a construction.
Provider	Use a theorem about the exact provider term.	Promote a passive abstract provider law to source evidence.
Sketch	Refute one completion, or prove that all completions violate the contract.	Reject a whole sketch because a single completion failed.

The finite companions cover parts of these mathematical and protocol models; they do not execute the actual frontend boundary. The newly compiled Karst, Schist, and Moraine files provide a few genuine Lean components. Each test should state whether it checks a theorem, a rejected proof attempt, an adapter refusal, or the display of an unresolved obligation. Failure of an unsupported certificate is not refutation of its target proposition.

The peer’s 48-family catalogue is a valuable extended backlog with per-report locators. It is not 48 executed language tests, and its 142 report–family incidences are not 142 distinct source rows. The 22 prioritized pairs above identify immediate implementation boundaries. When expanding a family, record its source locator, exact target, valid neighbor, a witness of the benchmark’s intended admissible premises, expected outcome, acceptance boundary, and current execution status. Vacuous propositions can be legitimate mathematics; the witness requirement protects a benchmark intended to exercise successful reasoning.

Expected failures must remain route-specific. An L^1 premise can be required by one interchange theorem without being necessary for every interchange. A sharpness claim needs a lower-bound argument; attainment is one sufficient route, while a sequence approaching the bound can also suffice. Do not turn the catalogue’s convenient diagnostics into universal mathematical necessities.

14.4 Semantic reading and stopping rules

Before showing the expanded proof, ask readers to reconstruct the compact statement: object, quantifiers, region, measure, branch, precision, completeness, and open conditions. Include plausible invalid neighbors with equally polished formatting. Then reveal the detailed view and measure whether it corrects misunderstanding. A smaller source file that makes readers infer a stronger theorem is not successful compression.

Predeclare improvement thresholds after a pilot estimates variability and before the main comparison. Reduce the new interface if its gains disappear against L2, its extra inference is unpredictable, semantic-reading errors rise, or maintenance costs exceed saved author effort. Continue with a library or service when that is where the evidence locates the benefit. No proposal can settle this choice by consensus alone.

15 Questions for the next iteration

The next discussion should choose concrete artifacts and acceptance criteria. These questions are intended to be answered by implementation and comparison, not by another unrestricted catalogue of language features.

1. **Which first use-site interface will we ship?** Is the minimum a property-aware binder and method application, or can existing Lean syntax plus an obligation view test the same hypothesis? Specify one exact-quotient example and its ordinary Lean control before adding more grammar.
2. **What is the finite automatic fragment?** Choose qualifier generation rules, supported theorem shapes, budgets, and the meaning of an unresolved frontier. Is relative completeness promised only after the ground rule set is fixed, as recommended here?
3. **What may inference decide about data?** Define which structures, witnesses, and representations are fixed by the source, which may be constructed from an authorized specification, and which ambiguity must be shown. Include a regression where an import changes available instances.
4. **How will required proof methods be checked?** Decide how a required reason differs from a hint, which dependencies are allowed, and whether a support-restricted helper theorem is the first implementation mechanism. Use the Frey arithmetic context to test contradiction shortcuts.
5. **Which behavioral bridge and completeness claim come first?** Choose the candidate grammar and admissible input domain. Finish the relevant normalization and reconstruction proofs; distinguish positive soundness, decision completeness, and realizable refutation, including `List Empty`.
6. **What authorizes negative Leant selection?** Require a concrete counterexample, a proved realization theorem, or a separate proof of the negation of the original specification. Specify behavior on unrealizable abstract witnesses and correlated outputs; test `List Empty` explicitly.
7. **Which nonalgebraic task is the first transfer test?** Choose between local analysis, summability/interchange, and coherent relational maps. Keep the task unseen until the interface and evaluation rules are fixed.
8. **What evidence is retained and rechecked?** Choose the proof, target reconstruction, provider-law dependencies, source correspondence, and policy information needed for replay. Exercise simplification, a dependent index change, a rebuilt local context, and a toolchain migration. When is reconstructing the index preferable to transporting its cached entries?
9. **What performance result would justify a larger checker or native execution?** Measure index size, rule firing, proof growth, rebuild cost, and single-lemma lookup separately from composition. State the checking-cost target and permitted axiom policy before adding a reifier or an accelerated route.
10. **What outcome would end the separate-language experiment?** Set criteria for keeping only the library, services, or editor interface. A useful next iteration could allocate three focused efforts—semantic bridge, authoring prototype, and independent evaluation—instead of nine overlapping new proposals.

A Convergence crosswalk

The ten columns correspond to the comparison dimensions in Section 3. An explicit mark means the report develops the policy, not that it implements it. A partial mark records a narrower treatment; absence would mean no support was located in the reviewed source. These are editorial judgments at the pinned snapshot. Source citations and qualifications for every cell are retained in `convergence.json`. Editorial marks: E = explicit, P = partial, A = absent. These describe the proposed designs; they do not score novelty or implementation.

Proposal	D1	D2	D3	D4	D5	D6	D7	D8	D9	D10
Basalt	E	E	E	E	E	E	E	E	E	E
Fiber	E	E	E	E	E	E	E	E	E	E
Gneiss	E	E	E	E	E	E	E	E	E	E
Karst	E	E	E	E	E	E	E	E	E	E
Moraine	E	E	E	E	E	E	E	E	E	E
Obsidian	E	E	E	P	E	E	E	E	E	E
Schist	E	E	E	E	E	E	E	E	E	E
Tephra	E	E	E	E	E	E	E	E	E	E
Trellis	E	E	E	E	E	E	E	E	E	E
Explicit	9	9	9	8	9	9	9	9	9	9
Partial	0	0	0	1	0	0	0	0	0	0
Absent	0	0	0	0	0	0	0	0	0	0

D1: conservative Lean elaboration; D2: same-object scoped evidence; D3: separate structure data and proof properties; D4: dependent behavioral composition and consequence; D5: bounded property inference; D6: explicit observation and transport guarantees; D7: proof-linked behavioral synthesis; D8: denotation, checker soundness, and original-target bridge; D9: statement, method, and kernel truth distinguished; D10: matched Lean baselines and stop rules.

Of 90 cells, 89 are explicit, one is partial, and none is absent. Obsidian explicitly develops behavioral application and dependent composition, but does not separately develop the full pre/postcondition consequence rule; its D4 mark is therefore partial under this compound criterion. All source locators and qualifications are retained in `convergence.json`. The shared prompt and prior syntheses limit the independence of these readings.

B Earlier questions, restated with current answers

The two round-2 syntheses asked overlapping questions. The following thematic crosswalk restates their substance so no prior report is needed. The current answers distinguish implemented evidence from unresolved design choices.

Earlier question	Current assessment
Which task must run from authored source to retained proof?	Exact quotient transport is a concrete property-use task; a length request tests behavioral acceptance. The seven checked cores are ingredients, not that complete frontend.
What must a computational result specify?	Its original object, relation, structures, guards, and requested strength. A refinement changes available evidence; an observation changes a view; a construction may change the witness.

Earlier question	Current assessment
How is a checker proved sound?	Define data and denotation, prove operation preservation and acceptance soundness, connect the original target, and justify execution. Schist now supplies this chain for a tiny affine example; broader reification remains open.
Where does existing <code>grobner</code> fit?	A bounded proof-producing control or service in its supported fragment. No completeness on failure or reusable external certificate format is assumed here.
What new workspace state is justified?	Demand and source indices, explanations, and replay/migration records around Lean's authoritative contexts and proofs. Measure the added state rather than duplicating it.
Should operations be partial or total by default?	Preserve imported Lean meaning. Offer partial-domain interfaces explicitly. A total function with a guarded theorem is not a function requiring a proof argument.
Which definition package should come first?	For the current hypothesis, start with the small arithmetic and behavioral interfaces. Formal-series jets, quotient towers, and rich analytic packages remain distinct candidates whose infrastructure cost must be measured.
What choices may the author delegate?	Routine proof search under a fixed proposition, and data construction under an explicit authorized specification. Structure changes, weakened conclusions, and hidden branch choices are not ordinary proof inference.
What conditional information remains visible?	Every unresolved condition and accepted distinction affecting domain, measure, branch, precision, completeness, or trust. Routine proofs may be folded; the guarantee may not be overstated.
What representations may change automatically?	Those with an applicable proved interface under the present context and requested relation. Exact equivalence, one-way observation, and new-witness construction require different adapters.
What negative conclusion may Leant report?	Distinguish search failure, unsupported fragment, abstract counterexample, concrete counterexample, and proved negation. Realization and specification transfer are required for promotion from the abstract case.
When does native execution earn its assumptions?	No threshold is established by these reports. Measure actual checker cost under a specified toolchain, and decide policy separately from speed.
What survives publication and upgrades?	The historical proof and environment record survive as historical evidence. Current reuse needs a successful check of the target, dependencies, and policy in the destination environment.
Can the mutation catalogue become executable?	Some mathematical and protocol cases now run in Python and a few in Lean. The proposed frontend, candidate reification, and source-display mutations still need actual integration tests.
What would justify keeping only the library and tools?	Gains vanishing against equally equipped Lean, increased semantic misunderstanding, or excessive repair and maintenance cost. This remains an intended legitimate outcome.
Is another set of nine designs useful?	The architectural convergence is now strong. Complementary implementation and independent evaluation work are more likely to resolve the remaining uncertainty.

C Validation and reproducibility

C.1 Retained and fresh executions, with their limits

All nine Python companions were inspected and run with bytecode disabled from isolated copies or redirected output directories. Commands, versions, copied source hashes, results, and unchanged-input checks are retained. Their counts have different units and should not be added into a fictitious theorem count.

Proposal	Retained Python result	Lean theorems
Basalt	24 test methods.	2
Fiber	1,298 finite checks in 31 groups.	3
Gneiss	32 test methods.	2
Karst	Eight groups, matching the original result JSON.	12
Moraine	32 tests; 25,600 interpreter comparisons and 729 affine-form pairs.	16
Obsidian	608 probes, matching the original result JSON.	—
Schist	10 test methods.	8
Tephra	23 test methods.	6
Trellis	31 cases: nine acceptance and 22 rejection.	—

All seven standalone Lean source files compile unchanged under Lean 4.32.0, commit [8c9756b](#) (complete identity in the receipts), using its bundled `Init`, `Std`, or `Lean` imports. No Lake command, repository build, dependency download, or output `.olean` was requested. The files contain 49 named theorem declarations, plus definitions and examples; this is a source inventory, not 49 independent completed design goals.

The retained axiom queries cover 32 unique declarations in those files: 27 report no axioms and five report only `propext`. The other 17 named theorems compiled but were not individually queried in this audit. The two additional synthesis interface theorems also compile and report no axioms. These inventories do not establish a full axiom audit of all proof-bearing definitions, a mechanically verified frontend, or a proof of the general multivariate checkers described in the prose.

C.2 The parallel review and this revision’s additional checks

The peer’s core logs contain 48 distinct axiom inventories, while the original files contain 49 named theorems. Its smaller headline omits Karst’s `viewIntro` from the declaration count because that declaration has no printed query. Our 49-declaration inventory and 32 individually audited original declarations remain distinct. The imported additional queries are historical evidence, not newly repeated runs of those cores.

This revision freshly accepts three further files. The unchanged peer `FreyExact.lean` contains six named theorems and seven examples, with four interface-theorem axiom queries. A modified `SupplyChecked.lean` accepts four examples: continuity, local derivative equality, matrix reversal, and inhabited Frey premises. It removes the intentional failing `Mvcgen` probe and uses the replacement matrix lemma. Both use existing Mathlib artifacts at dependency revision [81a5d25](#); no Lake or external build is run. The three new `AdequacyChecks.lean` theorems use bundled Lean and report no axioms. Source copies, exact commands, dependency revisions, outputs, and limits are retained separately under `evidence/parallel-review/`.

C.3 What the package verifier checks

The verifier checks Git and byte identity for the 83 original and 28 peer artifacts, the 90-cell crosswalk and source anchors, retained execution receipts, references and local links, three strict pdfLaTeX passes, source/PDF hashes, and the visual-review receipt for every page.

These consistency checks do not prove mathematical prose or a complete frontend. Only explicitly identified Lean declarations receive kernel-acceptance claims. The README gives dependencies and rerun commands; historical receipts retain their original scope, and the input packages remain unchanged.

References

- [1] Basalt. *A Property- and Behavior-Aware Mathematical Language over Lean*. Round-3 proposal, 2026. [Local article](#).
- [2] Fiber. *Refinement-Directed Mathematical Reasoning over Lean*. Round-3 proposal, 2026. [Local article](#).
- [3] Gneiss. *Property- and Behavior-Aware Mathematical Elaboration over Lean*. Round-3 proposal, 2026. [Local article](#).
- [4] Karst. *Scoped Refinements and Behavioral Contracts for Mathematical Proof*. Round-3 proposal, 2026. [Local article](#).
- [5] Moraine. Round-3 proposal on refinement-aware mathematical elaboration, 2026. [Local article](#).
- [6] Obsidian. *Property-Aware Types and Lawful Mathematical Reasoning over Lean*. Round-3 proposal, 2026. [Local article](#).
- [7] Schist. *A Property-Aware Mathematical Language over Lean*. Round-3 proposal, 2026. [Local article](#).
- [8] Tephra. Round-3 proposal on property-aware mathematical elaboration, 2026. [Local article](#).
- [9] Trellis. *Property-Rich Mathematics with Proof-Explicit Elaboration*. Round-3 proposal, 2026. [Local article](#).
- [10] Codex. *Mathematical Objects, Certified Computation*. Revised round-2 synthesis. [Local article](#).
- [11] *Nine Workbenches*. Revised round-2 synthesis. [Local article](#).
- [12] Claude (Anthropic). *Nine Refinements: Property-Aware Typing in a Mathematician’s Proof Language over Lean*. Parallel round-3 synthesis, 6 September 2026, imported at 8ddc6ab. [Local article](#); source; critical incorporation memo.
- [13] Fermat repository. Finite/free tensor-family representability, `P2M/Sol/S_Algebra_exists_alg Hom_equiv_pi.lean`, commit `aa2d8b3`; hypotheses on line 37. Pinned upstream source; Retained source.
- [14] Vladimir Reshetnikov et al. *Leant*. Bounded source review at commit `823259f`; complete identity in the linked source. Pinned synthesis sources.
- [15] Mathlib contributors. *Mathlib.Tactic.FunProp*. Official module documentation, consulted 6 September 2026. https://leanprover-community.github.io/mathlib4_docs/Mathlib/Tactic/FunProp.html.
- [16] Lean team. *The mvcgen tactic: Overview*. Official Lean Language Reference, consulted 6 September 2026. <https://lean-lang.org/doc/reference/latest/The--mvcgen--tactic/Overview/>.
- [17] Lean team. *Axioms*. Official Lean Language Reference; discussion of classical choice and parametricity. <https://lean-lang.org/doc/reference/latest/Axioms/>.